

FLUTTER MEDELLÍN • 2026

Del Code Review al Code Prevention

Porque el mejor code review es el que nunca llega al PR.

Andrés Angulo @andresanhi

```
analysis_options.yaml  
include: package:flutter_lints/flutter.yaml  
analyzer: strict-inference: true  
linter: rules: avoid_print: true
```

Lo que vamos a ver

-
- | | | |
|----|---------------------------------|--|
| 01 | El costo del code review | Cuando el feedback llega demasiado tarde |
|----|---------------------------------|--|
-
- | | | |
|----|--------------------------|-----------------------------|
| 02 | Análisis estático | Shift left, Sonar y linters |
|----|--------------------------|-----------------------------|
-
- | | | |
|----|-------------------------|-----------------------------------|
| 03 | Lo que ya tienes | flutter_lints y el SDK de Flutter |
|----|-------------------------|-----------------------------------|
-
- | | | |
|----|------------------------------|---------------------------------------|
| 04 | Más allá del estándar | very_good_analysis, DCM y 450+ reglas |
|----|------------------------------|---------------------------------------|
-
- | | | |
|----|---------------------------|---|
| 05 | Tus propias reglas | AST, custom_lint y convenciones de equipo |
|----|---------------------------|---|
-
- | | | |
|----|---------------------|--|
| 06 | Linters e IA | Por qué el linter gana cuando todo es generado |
|----|---------------------|--|
-

¿Cuánto tiempo de un code review
usamos en cosas que
una herramienta pudo detectar
antes?

EL PROBLEMA

Revisando un PR cualquiera

user_data.dart ⚠ 3

```
1  import 'package:flutter/material.dart' ;
2  import 'package:http/http.dart' ;
3
4  class UserData extends StatelessWidget {
5      final String name ;
6      final Color primary = Color ( 0xFF1976D2 );
7
8      String unusedVar = 'debug' ;
9
10     @override
11     Widget build ( BuildContext context ) {
12         return Container ();
13     }
14 }
```

✗ unused_import · línea 2
⚠ avoid_hardcoded_color · línea 6
✗ unused_local_variable · línea 8

Nada de esto era lógica de negocio. Era ruido que una herramienta pudo haber resuelto antes.

LOS DATOS

No es solo tu equipo

50%

Del tiempo de review se va en
formato, nombres y estilo

[Code Review Guidelines — Petar Ivanov](#)

38%

De las reviews estudiadas
recibió feedback sobre
convenciones

[Microsoft Research — Allamanis et al.](#)

65%

De los equipos está
insatisfecho con su proceso de
review

[SmartBear — State of Code Review Survey](#)

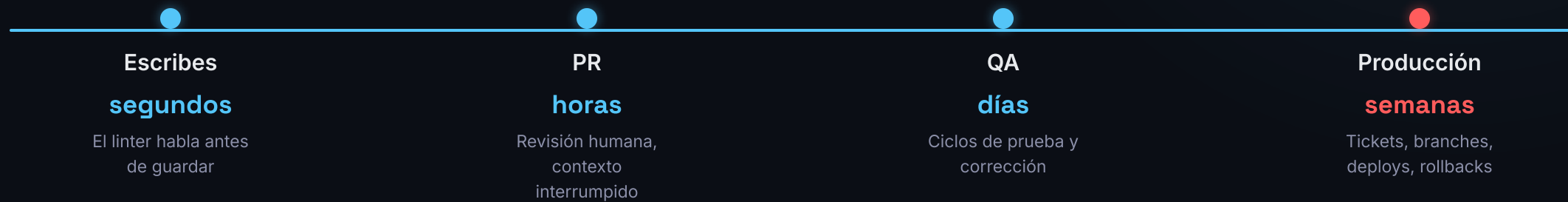
¿Y si **la mitad** de esos comentarios
nunca hubiera llegado al PR?

FUNDAMENTOS

El error más caro es el que llega tarde.

Shift left — mover la detección de problemas al inicio del ciclo, cuando corregirlos cuesta casi nada.

Análisis estático — es la forma más efectiva de lograrlo: revisa tu código sin ejecutarlo, mientras escribes.



El análisis estático revisa tu código **sin ejecutarlo** — como un corrector que entiende sintaxis, tipos y contexto. Su objetivo es mover el problema a la izquierda.

HERRAMIENTAS

Análisis estático en la práctica

PROYECTO SonarQube / SonarCloud

Plataforma de análisis que corre en tu pipeline de CI. Ve el proyecto completo — no un archivo, sino la evolución del código en el tiempo.

Deuda técnica

Security hotspots

Tendencias

Quality gates

vs

LOCAL Linters en el IDE

Herramienta que vive en tu editor. Te da feedback mientras escribes — antes de compilar, antes de hacer commit, antes de que nadie más lo vea.

Feedback inmediato

Costo casi cero

Sin pipeline

Mientras tipeas

No son competidores. El linter te habla a **ti**. Sonar le habla al **equipo**.

Ya tienes más de lo que crees

01

Dart Analyzer

El motor nativo del SDK — activo desde el primer proyecto

02

Reglas de la comunidad

`flutter_lints` · `very_good_analysis` · DCM

03

Reglas propias

Las convenciones de tu equipo, codificadas y automatizadas

Cada capa tiene su nivel de poder. *¿En cuál estás tú?*

Lo que Flutter ya te da

`dart analyze`

El motor nativo del SDK. Corre en segundo plano mientras escribes, sin configuración adicional.

`analysis_options.yaml`

El archivo de configuración presente en todo proyecto Flutter nuevo. Es el punto de entrada para personalizar el análisis.

`flutter_lints`

El paquete oficial de Google con las reglas básicas recomendadas — incluido por defecto desde Flutter 2.5.

Sin instalar nada. Sin configurar nada. Ya estás protegido en el nivel más básico.

Tu primer contrato de calidad

EXPLORER

▼ flutter_linters_demo

▶ .dart_tool

▶ android

▶ ios

▶ lib

▶ test

! analysis_options.yaml

≡ pubspec.yaml

! analysis_options.yaml

```
# This file configures the analyzer
# which statically analyzes Dart code.
#
# The following line activates recommended lints:
include: package:flutter_lints/flutter.yaml
```

linter:

rules:

avoid_print: false

prefer_single_quotes: true

avoid_print: true # No prints en producción

prefer_const_constructors: true # Mejor rendimiento

prefer_single_quotes: true # Consistencia de estilo

require_trailing_commas: true # Mejor diff en PRs

15 minutos de configuración. Cero debates de estilo en el próximo PR.

Humano, pero mejorado.

`very_good_analysis`

El estándar de Very Good Ventures para proyectos de escala. Más de 100 reglas adicionales sobre `flutter_lints` — tipado estricto, imports absolutos, manejo de errores tipado.

100+

reglas adicionales activas con un solo cambio en el `include`

`dart_code_metrics` / DCM

Va más allá del estilo — mide la salud del proyecto. Complejidad ciclomática, código muerto, dependencias mal usadas entre capas de arquitectura.

100

reglas en el plan gratuito · 500+ en el plan pro

`dart_code_linter` / DCL MIT · Gratuito

Fork de Dart Code Metrics mantenido por **Bancolombia** — sí, el banco colombiano. Open source, licencia MIT, con Quick Fixes incluidos.

70+

reglas prebuilt · completamente gratuito · actualizado hace 3 días

No reemplaza a Terrícola — lo extiende. Y los tres pueden coexistir en el mismo proyecto.

very_good_analysis

```
≡ pubspec.yaml
```

```
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  very_good_analysis: ^6.0.0
```

```
! analysis_options.yaml
```

```
# Reemplaza flutter_lints por very_good_analysis  
include: package:very_good_analysis/analysis_options.yaml  
  
linter:  
  rules:  
    # Imports absolutos – evita rutas relativas frágiles  
    always_use_package_imports: true  
    # Prohíbe llamadas a tipo dynamic – más seguridad de tipos  
    avoid_dynamic_calls: true  
    # Parámetros booleanos siempre nombrados – más legibilidad  
    avoid_positional_boolean_parameters: true
```

Un solo cambio en el `include` . Más de 100 reglas activas desde ese momento.

Algunas de las 100+ reglas

`always_use_package_imports`

Imports absolutos, nunca relativos

`avoid_dynamic_calls`

Prohibe llamadas a tipo dynamic

`prefer_constructors_over_static_m...`

Mejor legibilidad en constructores

`avoid_positional_boolean_paramete...`

Booleanos siempre nombrados

`no_adjacent_strings_in_list`

Evita strings concatenados accidentalmente

`prefer_named_parameters`

Claridad en llamadas a funciones

`avoid_catches_without_on_clauses`

Manejo de errores tipado

`use_decorated_box`

Rendimiento en widgets decorados

`avoid_unused_constructor_paramete...`

Limpieza de constructores

`require_trailing_commas`

Mejor diff en PRs

100+ reglas activas. Solo con cambiar el `include`.

Dart Code Metrics

GRATUITO

CLI + reglas básicas

Análisis local, métricas de complejidad y código muerto

PRO

450+ reglas + CI/CD

Integración con pipelines, reportes avanzados y soporte premium

```
≡ pubspec.yaml
```

```
dev_dependencies:  
  dart_code_metrics: ^5.0.0
```

```
! analysis_options.yaml
```

```
analyzer:  
  plugins:  
    - dart_code_metrics  
dart_code_metrics:  
  # Complejidad máxima permitida por función  
  metrics:  
    cyclomatic-complexity: 20  
  # Reglas específicas de DCM  
  rules:  
    avoid-unused-parameters: true  
    no-empty-block: true
```

DCM no reemplaza `very_good_analysis` — los puedes tener activos al mismo tiempo. Uno va en el `include`, el otro como `plugin`. Juntos cubren estilo, arquitectura y métricas de salud del proyecto.

Algunas de las 450+ reglas de DCM

`avoid-unused-parameters`

Detecta parámetros declarados pero nunca usados

`no-empty-block`

Prohíbe bloques vacíos sin comentario explicativo

`prefer-extracting-callbacks`

Extrae callbacks inline a métodos nombrados

`avoid-nested-conditional-expressi...`

Limita ternarios anidados

`no-magic-number`

Prohíbe números literales sin constante nombrada

`prefer-match-file-name`

El nombre del archivo debe coincidir con la clase principal

`avoid-returning-widgets`

Evita retornar widgets desde métodos — usa clases

`avoid-unnecessary-setstate`

Detecta setState innecesarios

`prefer-single-widget-per-file`

Un widget público por archivo

`cyclomatic-complexity`

Limita la complejidad lógica por función

450+ reglas especializadas en Flutter y Dart. Muchas que `very_good_analysis` no cubre.

Las reglas que solo tu equipo necesita

`very_good_analysis`

Cubre estilo y buenas prácticas generales. No sabe que en tu equipo todos los servicios deben terminar en `ServiceImpl`.

`dart_code_metrics`

Cubre métricas y complejidad. No sabe que en tu proyecto los colores solo pueden venir del Design System.

¿Y las convenciones que son *solo tuyas*?

`custom_lint`

Las convenciones de tu equipo, codificadas como reglas. Automáticas, inmediatas y sin depender de que alguien las recuerde.

Ningún paquete externo conoce las decisiones de tu equipo. **custom_lint** te da el poder de codificarlas.

Cómo el analizador entiende tu código

El analizador no lee tu código como texto plano — construye un **Árbol de Sintaxis Abstracta (AST)**: una representación estructurada donde cada elemento tiene un nodo con significado preciso.

Abstract

No representa cada carácter — solo la estructura lógica del programa.

Syntax

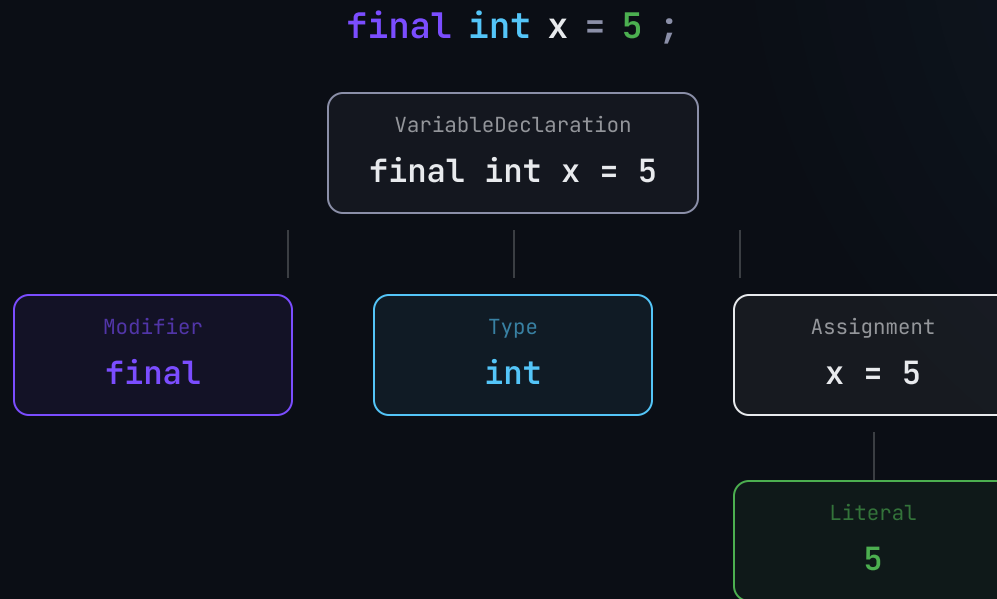
Refleja las reglas gramaticales del lenguaje — tipos, declaraciones, expresiones.

Tree

Cada nodo puede tener hijos. La raíz es el archivo completo.

El linter no busca texto — recorre nodos. Por eso puede entender el contexto, no solo las palabras.

Cómo ve el analizador tu código



El linter hace lo mismo que aprendiste en primaria:

El equipo escribe código limpio

Sujeto

Verbo

Predicado

El Visitor — escucha, no busca

♦ repository_naming_rule.dart

```
class RepositoryNamingRule extends DartLintRule {
  # Define el código de error que se mostrará
  static const _code = LintCode(
    name: 'repository_naming',
    problemMessage: 'Must end with RepositoryImpl',
  );

  # El Visitor — solo escucha ClassDeclaration
  @override
  void run (CustomLintResolver resolver,
    ErrorReporter reporter,
    CustomLintContext context) {
    context.registry.addClassDeclaration((node) {
      final name = node.name.lexeme;
      if (!name.endsWith('RepositoryImpl')) {
        reporter.reportErrorForNode(_code, node.name);
      }
    });
  }
}
```

EL VISITOR SOLO REACCIONA A:

- ✓ ClassDeclaration Escucha este nodo
- MethodInvocation Lo ignora
- ✓ ClassDeclaration Evalúa el nombre
- StringLiteral Lo ignora
- ✗ UserRepository Reporta el error

Tu convención, tu regla

Toda implementación de `Repository` debe terminar en `RepositoryImpl` — la convención del equipo, ahora automatizada.

✗ Viola la convención

```
class UserRepository implements Repository<User> {  
    // ...  
}
```

✗ repository_naming: Must end with RepositoryImpl



✓ Convención respetada

```
class UserRepositoryImpl implements Repository<User> {  
    // ...  
}
```

✓ Sin errores

Esta regla no existía en ningún paquete. La escribió tu equipo una vez — ahora ningún PR puede violarla.

Del warning al fix automático

♦ repository_naming_rule.dart

```
# 1. Define el fix junto a la regla
@override
List<Fix> getFixes() => [_AddImplSuffix()];

# 2. El fix calcula dónde y qué cambiar
class _AddImplSuffix extends DartFix {
  @override
  void run(CustomLintResolver resolver,
    ChangeBuilder builder, ...) {
    builder.addDartFileEdit((edit) {
      edit.addSimpleInsertion(
        node.name.end, 'Impl',
      );
    });
  }
}
```

♦ user_repository.dart

```
class UserRepository implements Repository<User> {
  // ...
}
```

💡 Quick Fix

✓ Rename to `UserRepositoryImpl`

```
class UserRepositoryImpl implements Repository<User> {
  // ...
}
```

✓ Sin errores — fix aplicado

El linter detecta. El Quick Fix corrige. Y si nadie hace caso — `dart run custom_lint` en el pipeline lo bloquea antes del merge.

La IA escribe código más rápido que
nunca.

¿Quién lo revisa?

Velocidad no es estabilidad

8×

más bloques duplicados en repositorios de Google, Microsoft y Meta en 2024

[GitClear, 2025](#)

39–44%

de brecha entre productividad percibida y real con IA

[METR, 2025](#)

7.2%

menos estabilidad por cada 25% más adopción de IA

[Google DORA, 2024](#)

Esto no es anti-IA. Es pro-calidad.

No necesitas un agente para esto

Cuando tu equipo usa Copilot, Cursor o Claude Code, el código generado no conoce tus convenciones. La reacción común es crear un agente, una skill, un prompt elaborado con todas las reglas.

Pero ya tienes esto:

01 `analysis_options.yaml`
Documentación ejecutable — dáselo al agente como contexto

02 `flutter analyze --fatal-warnings`
El gate que valida todo el código generado sin un token adicional

03 `custom_lint`
Tus reglas de equipo, activas para cualquier código — humano o generado

EMPIEZA MAÑANA

Elige tu nivel. Empieza hoy.



TERRÍCOLA

El estándar

`flutter_lints` + `analysis_options.yaml`

15 minutos



ANDROIDE

Mejorado

`very_good_analysis` + DCM + CI

1 hora



SAIYAJIN

Sin límites

`custom_lint` + reglas de tu equipo

Un par de semanas

Cada comentario que evitas con un linter es tiempo que recuperas para **construir producto**.

¿Preguntas?

Gracias



Slides y recursos



Conectemos

@andresanhi · github.com/andresanhi